Uruchomienie ai-context-bridge

Instrukcja dla macOS, Windows 11 i Linux

Checkpointy pracy oparte na faktach z Git i opcjonalnym szkicu od modelu. Przejdź od wymagań systemowych do pierwszego pause i resume. Narzędzie nie uruchamia usługi w tle.

W skrócie

- Python 3.11 lub nowszy oraz Git.
- pause zapisuje plik **.bridge/checkpoint.md** w repozytorium.
- Bez konfiguracji LLM działa tryb tylko faktów i ręczny opis następnego kroku.
- Model dostaje tylko nazwy zmienionych plików i komunikaty commitów.

Zacznij od repozytorium
Przygotuj lokalny checkout projektu ai-context-bridge. Polecenia instalacyjne wykonuj w jego katalogu głównym, obok pliku pyproject.toml.

1. macOS

Otwórz Terminal. Z Homebrew zainstaluj Python i Git:

```
brew install python@3.11 git
python3 --version
git --version
```

Jeśli nie masz Homebrew, instrukcja jest na brew.sh. Git można też zainstalować narzędziami Apple: **xcode-select --install**. W katalogu projektu:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install .
bridge --help
```

2. Windows 11

Otwórz PowerShell i zainstaluj wymagania przez winget:

```
winget install Python.Python.3.11
winget install Git.Git
```

Po instalacji uruchom ponownie PowerShell. Możesz też użyć instalatorów z python.org oraz Git for Windows. Sprawdź wersje:

```
py -3.11 --version
git --version
```

W katalogu projektu utwórz środowisko i zainstaluj program:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install .
bridge --help
```

Jeśli aktywacja jest zablokowana, nie zmieniaj ustawień systemowych - uruchom program przez **.\.venv\Scripts\bridge.exe**.

3. Linux

Ubuntu/Debian:

```
sudo apt update
sudo apt install python3 python3-venv git
python3 --version
git --version
```

Jeśli Python jest starszy niż 3.11, zainstaluj aktualną wersję według dokumentacji dystrybucji. W Fedora użyj **sudo dnf install python3 git**. W katalogu projektu:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install .
bridge --help
```

4. Zapisz i wznów pracę

Wskaż repozytorium Git (ścieżka może być względna lub bezwzględna):

```
bridge pause --repo /ścieżka/do/repo
bridge resume --repo /ścieżka/do/repo
```

W PowerShell przykładowa ścieżka to **C:\Users\Ala\projekty\moja-aplikacja**. Możesz pominąć --repo, aby użyć bieżącego katalogu.

> **Podczas pause**
> Program pokaże szkic, jeśli LLM jest skonfigurowany, i zapyta „Co jest następnym krokiem?”. Enter zatwierdza szkic; wpisany tekst go zastępuje. Ścieżka zapisu .bridge/checkpoint.md pojawi się w terminalu.

5. LLM, offline i prywatność

Bez konfiguracji LLM

Bez kompletu trzech zmiennych bridge nie wykona połączenia. Zapisze fakty z Git i pozwoli wpisać krok ręcznie.

Opcjonalny endpoint Chat Completions zgodny z OpenAI

Przykład dla lokalnego serwera zgodnego z API OpenAI, np. Ollama. macOS/Linux, bieżąca sesja:

```
export BRIDGE_BASE_URL="http://localhost:11434/v1"
export BRIDGE_API_KEY="ollama"
export BRIDGE_MODEL="nazwa-modelu"
```

Windows PowerShell:

```
$env:BRIDGE_BASE_URL = "http://localhost:11434/v1"
$env:BRIDGE_API_KEY = "ollama"
$env:BRIDGE_MODEL = "nazwa-modelu"
```

Ustaw wartości wymagane przez swój serwer. Nie zapisuj klucza w repozytorium; bridge go nie wypisuje. Błędna odpowiedź JSON jest ponawiana raz, potem checkpoint przechodzi w tryb tylko faktów.

Dry-run: sprawdź żądanie bez wysyłania

```
bridge pause --repo /ścieżka/do/repo --dry-run
```

Dry-run wypisuje treść żądania, nie łączy się z siecią i nie tworzy checkpointu. Do modelu wysyłane są tylko nazwy zmienionych plików i komunikaty ostatnich commitów. Bez diffów, treści plików, hashy, brancha, statusów Git, stashy i checkpointu.

Częste problemy

Nie znaleziono bridge	Aktywuj .venv ponownie albo uruchom program przez ścieżkę do pliku w .venv.
Katalog nie jest repo	W --repo wskaż repozytorium Git lub katalog wewnątrz niego.
Brak zapisu	Sprawdź uprawnienia; plik trafia do .bridge/checkpoint.md.

Miejsce zapisu
Checkpoint pozostaje lokalnie. bridge nie modyfikuje automatycznie .gitignore ani wykluczeń Git.